Part 12

MVC & MVT in Django

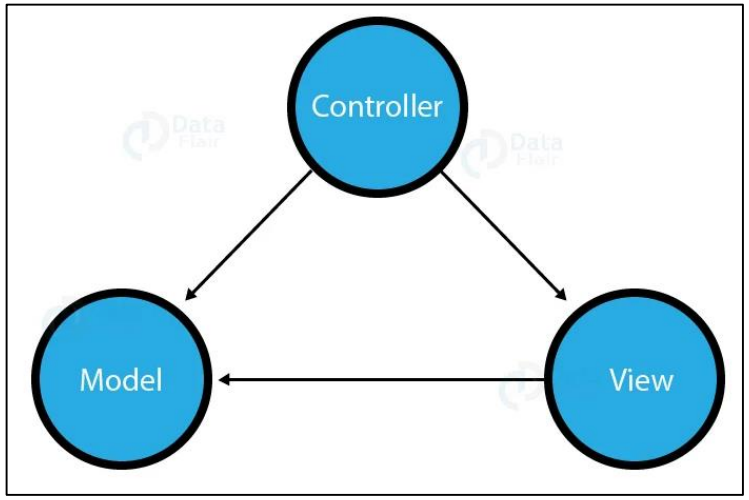
VERSION: 4.0.1

WRITTEN AND DESIGN BY: BABAR ALI

What is MVC

MVC pattern is a Product Development Architecture. It solves the traditional approach's drawback of code in one file, i.e., that MVC architecture has different files for different aspects of our web application/ website. The MVC pattern has three components, namely **Model**, **View**, and **Controller**.

This difference between components helps the developer to focus on one aspect of the web-app and therefore, better code for one functionality with better testing, debugging and scalability.



1. Model

The Model is the part of the web-app which acts as a mediator between the website interface and the database. In technical terms, it is the object which implements the logic for the application's data domain. There are times when the application may only take data in a particular dataset, and directly send it to the view (UI component) without needing any database then the dataset is considered as a model.

For example:

When you sign up on any website you are actually sending information to the controller which then transfers it to the models which in turn applies business logic on it and stores in the database.

2. View

This component contains the UI logic in the Django architecture.

View is actually the User Interface of the web-application and contains the parts like HTML, CSS and other frontend technologies. Generally, this UI creates from the Models component, i.e., the content comes from the Models component.

For example:

When you click on any link or interact with the website components, the new webpages that website generates is actually the specific views that stores and generates when we are interacting with the specific components.

3. Controller

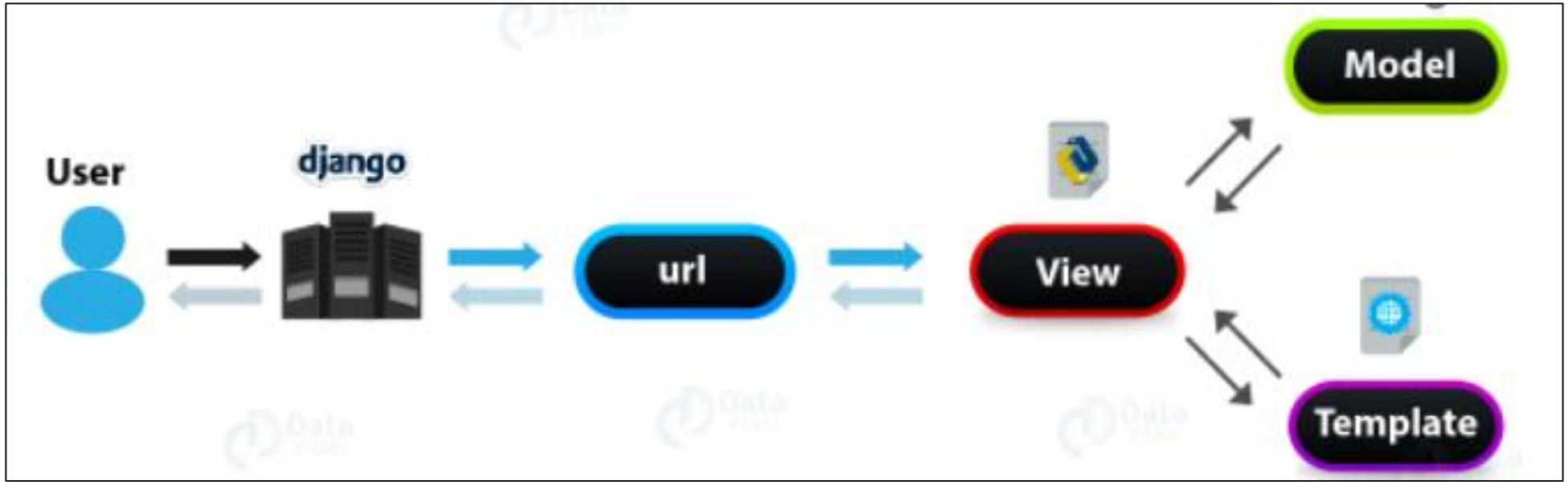
The controller as the name suggests is the main control component. What that means is, the controller handles the user interaction and selects a view according to the model.

The main task of the controller is to select a view component according to the user interaction and also applying the model component.

For example:

When we combine the two previous examples, then we can very clearly see that the component which is actually selecting different views and transferring the data to the model's component is the controller.

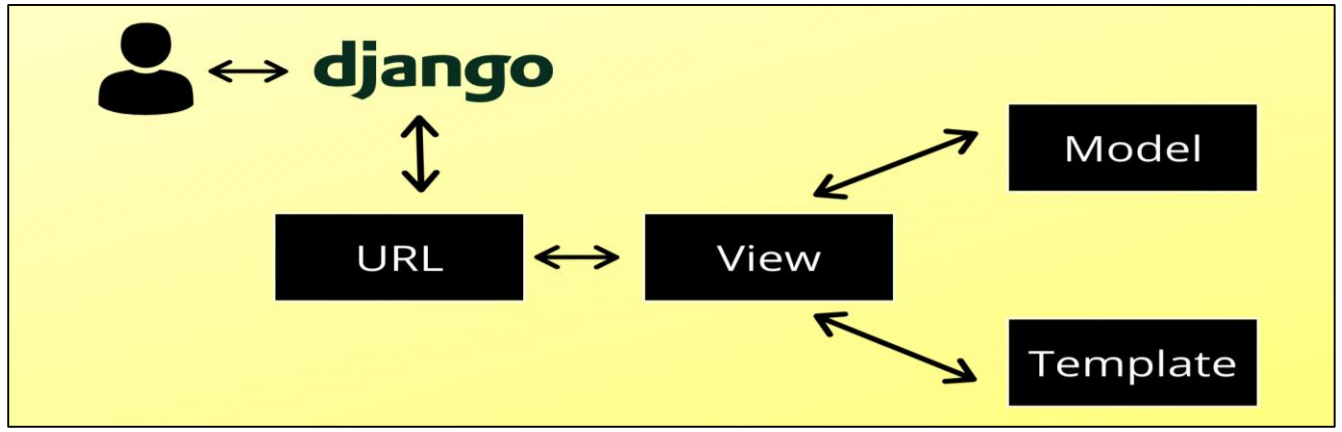
How MVC Work



Intro to MVT

What is MVT.?

MVT stands for Model-View-Template. Sometimes it is also referred to as MTV(Model-Template-View). MVT is a design pattern or design architecture that Django follows to develop web applications. It is slightly different from the commonly known **MVC(Model-View-Controller)** design pattern. controller



Django MVT

- The **Model** manages the data and is represented by a database. A model is basically a database table.
- The **View** receives HTTP requests and sends HTTP responses. A view interacts with a model and template to complete a response.
- The **Template** is basically the front-end layer and the dynamic HTML component of a Django application.

Django MVT

If we break down a Django project we will see its main components are:

1) URL-patterns
2) Views
3) Models
4) Templates

1. URL-Patterns

URL patterns contain the URLs a user will request for. Let's suppose we have a blog with the domain name www.ablog.com. The URL patterns of the blog might be www.ablog.com/posts or www.ablog.com/posts/post-1 or www.ablog.com/about etc. URL patterns are simply the requests of different pages of a web application.

A user will send a request to the server through a browser. After receiving a request Django searches if there is any corresponding response for this particular request. And this is where MVT starts to work.

2. Views

A **view** is a corresponding response to a request. Suppose a user sent a request www.ablog.com/posts/post-1. Django will search if there is a view for this request. If there is no view for this request then there is no response.

View can be a python function or a python class based on how we write our views. A view fetches data from models and renders a template. So mainly a view talks to models and templates to send responses to a request.

View basically works as the connection point for model and template. It also executes the logic of the response. We write our views inside a python file called views.py.

3. Model

A model is nothing but a database table. It is a python class that is linked to a database. When we create a model with some data we are actually saying to Django to create a database table for those data. So a model contains necessary data for a particular request. A view fetches necessary data from its corresponding model.

For the request, www.ablog.com/posts/post-1 we will write a model which will have the necessary data like the author of the post, date of creation, heading, subheading, main content, etc. We write models in a python file called models.py.

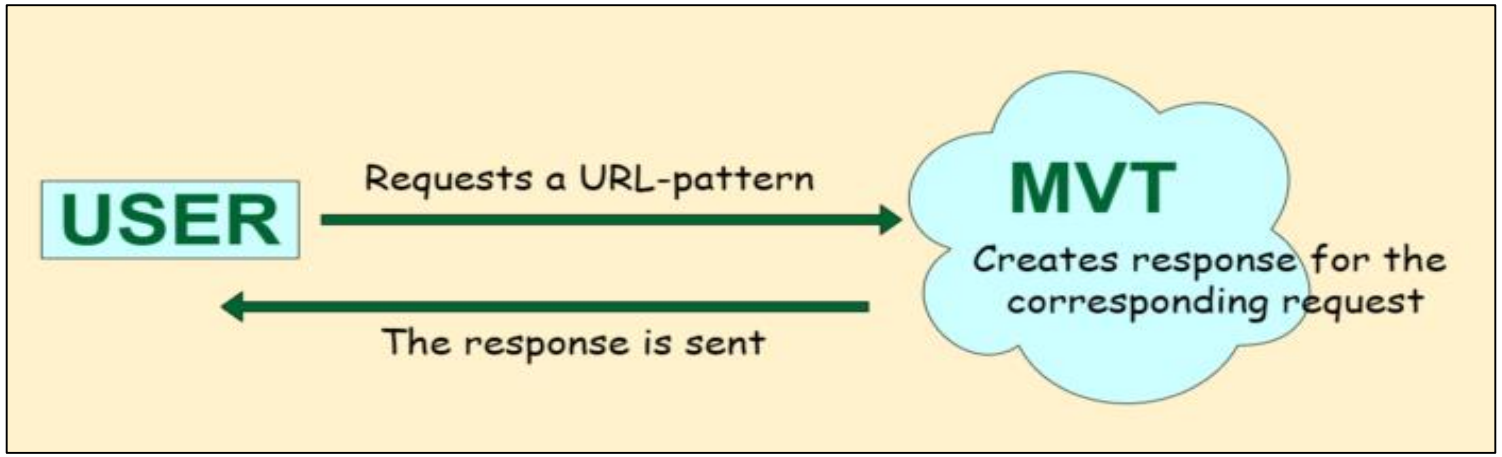
4. Template

I also said that a view renders a template for a particular request. A template is nothing but the front-end components of a Django application. It contains the static HTML output of the webpage as well as the dynamic information.

To generate HTML dynamically Django uses DTL(Django Template Language) along with static HTML. Using DTL Django is able to show data from models dynamically.

Conclusion

A user searches for a URL pattern that is nothing but a request for a certain web page. After sending the request to the server Django will search for the corresponding view for this request. View will talk to model and template to create a complete response for the request. So mainly the MVT structure together creates a response for a corresponding request.



Note

For every website on the Internet, there are 3 main components or code partitions; Input Logic, Business Logic, and UI Logic.

These partitions of code have specific tasks to achieve, the input logic is the dataset and how the data gets to organize in the database. It just takes that input and sends it to the database in the desired format. The Business logic is the main controller which handles the output from the server in the HTML or desired format. The UI Logic as the name suggests are the HTML, CSS and JavaScript pages.

Thanks for Reading